

Test Cases

PS-05 - RCA Drafter | Functional and integration test coverage

Coverage Overview

Category	Tests	Status
API Endpoints	6	PASS
Agent Pipeline	5	PASS
Input Validation	4	PASS
Error Handling	4	PASS
Output Format	3	PASS
Total	22	ALL PASS

1. API Endpoint Tests

ID	Test Name	Action / Input	Expected Result	Status
TC-001	Health Check	GET /api/health	HTTP 200 + {"status":"ok"}	PASS
TC-002	Generate RCA - valid input	POST /api/generate-rca with all 4 fields populated	success=true, rca field is non-empty Markdown string	PASS
TC-003	Generate RCA - missing field	POST /api/generate-rca without error_logs field	HTTP 400 + {"error":"Missing required fields"}	PASS
TC-004	Get all agents	GET /agents	List of 5 agents: coordinator, timeline, log_analysis, code_analysis, synthesis	PASS
TC-005	Create incident	POST /incidents with title, severity, timeline, error_logs, git_diff	HTTP 201 + incident object containing id and status="open"	PASS
TC-006	Trigger RCA on incident	POST /incidents/{id}/trigger-rca on existing incident	success=true, incident.status updated to "resolved", rca field populated	PASS

2. Agent Pipeline Tests

ID	Test Name	Action / Input	Expected Result	Status
TC-007	Timeline Agent	Pass multi-line timeline string with HH:MM timestamps	Returns list of event dicts each with timestamp and event keys	PASS

ID	Test Name	Action / Input	Expected Result	Status
TC-008	Log Analysis Agent	Pass raw block containing ERROR, WARN, INFO, FATAL lines	Returns only ERROR/FATAL entries; INFO lines excluded	PASS
TC-009	Code Analysis Agent	Pass git diff with added (+) and removed (-) lines	Returns dict with added_lines, removed_lines, and file_path	PASS
TC-010	Synthesis Agent	Pass outputs from all 3 analysis agents	Returns Markdown containing Root Cause, Timeline, and Action Items sections	PASS
TC-011	Coordinator full pipeline	Pass complete incident payload to CoordinatorAgent.generate_rca()	Final Markdown is non-empty; all 5 agent statuses return to idle	PASS

3. Input Validation Tests

ID	Test Name	Action / Input	Expected Result	Status
TC-012	Empty timeline field	Submit with timeline="" (logs and diff provided)	RCA generated from logs and diff only; no crash or 500 error	PASS
TC-013	Empty git diff field	Submit with git_diff="" (timeline and logs provided)	RCA generated from timeline and logs; no crash	PASS
TC-014	All input fields empty	Submit with all 3 evidence fields empty strings	HTTP 400 returned with a clear validation error message	PASS
TC-015	Very large log input	Submit a 10,000-line log file as error_logs	Response received within 60s timeout; no 500 or OOM error	PASS

4. Error Handling Tests

ID	Test Name	Action / Input	Expected Result	Status
TC-016	Ollama service offline	Stop Ollama, then POST /api/generate-rca with valid data	HTTP 500 + error message referencing Ollama or model connection	PASS
TC-017	Invalid incident ID	GET /incidents/nonexistent-id-xyz	HTTP 404 + {"error": "Incident not found"}	PASS
TC-018	Malformed JSON body	POST /api/generate-rca with Content-Type: application/json but invalid JSON	HTTP 400 returned; Flask server does not crash	PASS
TC-019	Malformed log lines	Submit logs with no timestamps or structure	Parser returns empty list gracefully; no exception raised in agent	PASS

5. Output Format Tests

ID	Test Name	Action / Input	Expected Result	Status
TC-020	RCA contains all required sections	Run full pipeline using Sample Incident 1 data	Markdown output includes: Summary, Timeline, Root Cause, Contributing Factors, Impact, Resolution, Action Items	PASS
TC-021	Severity classification accuracy	Submit Sample 1 (DB pool exhaustion, P1-level impact)	Severity field in output equals "P1"	PASS
TC-022	Action items are evidence-grounded	Run full pipeline using Sample Incident 1 data	Each action item references a specific service name, log line, or code change from the input	PASS

All test cases executed against the running application at <http://127.0.0.1:5000> with Ollama phi3:mini model active. Happy-path cases use Sample Incident 1 data from the Sample Data document.